

PNAAT Gestures ESP32

Home Automation with Embedded Artificial Intelligence

Manual gesture recognition system using a neural network
executed 100% on an ESP32-S3, with LED control via PWM,
NTP-synchronized clock, OLED display, and MQTT telemetry/remote control.

Warning: This is an academic capstone/final course project (academic prototype). It may contain inconsistencies, unhandled edge cases, and typical limitations of a learning project — it is not a production-ready product.

Platform: ESP32-S3 (Heltec WiFi LoRa 32 V3)
Framework: ESP-IDF v5.4.4
License: MIT

August 24, 2026

Contents

1	Features	2
2	Hardware and Pinout	2
3	Software Architecture	2
3.1	Components (<code>components/</code>)	2
3.2	Main Modules (<code>main/</code>)	3
3.3	Boot Sequence	4
3.4	Tasks	4
3.5	Partitions (<code>partitions.csv</code>)	4
4	Machine Learning Pipeline	4
5	Training Details (Edge Impulse)	5
5.1	Impulse Configuration (Windowing)	5
5.2	Signal Processing (DSP)	5
5.3	Neural Network Architecture	6
5.4	Results (Validation Set)	6
6	LED Control via PWM	6
7	OLED Display and NTP Clock	6
8	IoT / MQTT Connectivity	7
8.1	Publishing	7
8.2	Subscribing	7
8.3	Hybrid Semantics	8
8.4	Resilience	8
9	Built-in Diagnostics	9
10	How to Build and Flash	9
11	Technical Challenges Solved	10
12	Repository Structure	10
13	Technologies Used	11
14	Credits and Acknowledgments	11
15	License	11

1 Features

- Recognition of **5 gestures** by an embedded neural network: **right**, **left**, **up**, **down**, **idle**.
- **right** turns the LED on; **left** turns it off.
- **up/down** increase/decrease brightness in **20%** steps via PWM (LEDC, 5 kHz, 10 bits = 1024 levels).
- OLED SSD1306 128x64 display showing, in this order: time, gesture, LED, and brightness.
- Clock synchronized via **SNTP**, Brasília timezone (UTC-3).
- JSON MQTT telemetry on every action.
- Remote LED control via MQTT (**on/off/up/down**).
- Resilient offline operation, with automatic Wi-Fi reconnection.
- Built-in terminal diagnostics (sampling rate and INT pin pulse count).

2 Hardware and Pinout

Board: Heltec WiFi LoRa 32 V3 (ESP32-S3)

Sensors/peripherals: IMU BNO085 (breakout GY-BNO08X), onboard OLED SSD1306 128x64 (I2C address 0x3C), onboard white LED (GPIO35).

Component	Signal	Pin	Notes
BNO085	SDA	GPIO6	I2C_NUM_0, 100 kHz
BNO085	SCL	GPIO7	I2C_NUM_0, 100 kHz
BNO085	INT	GPIO5	Mandatory — signals new data
BNO085	RST	GPIO4	Fallback: software reset
BNO085	ADD	GND	Sets address to 0x4A
BNO085	VCC	3.3V fixed	Never power from the Vext rail
OLED	SDA	GPIO17	Dedicated I2C_NUM_1
OLED	SCL	GPIO18	Dedicated I2C_NUM_1
OLED	RST	GPIO21	—
OLED	Power	VEXT Rail (GPIO36)	Active LOW, must be enabled before display initialization
Console	TX/RX	GPIO43/GPIO44	UART

The project uses the **new I2C API** from ESP-IDF (`driver/i2c_master.h`), and not the legacy API.

3 Software Architecture

Framework: **ESP-IDF v5.4.4** (C/C++).

3.1 Components (components/)

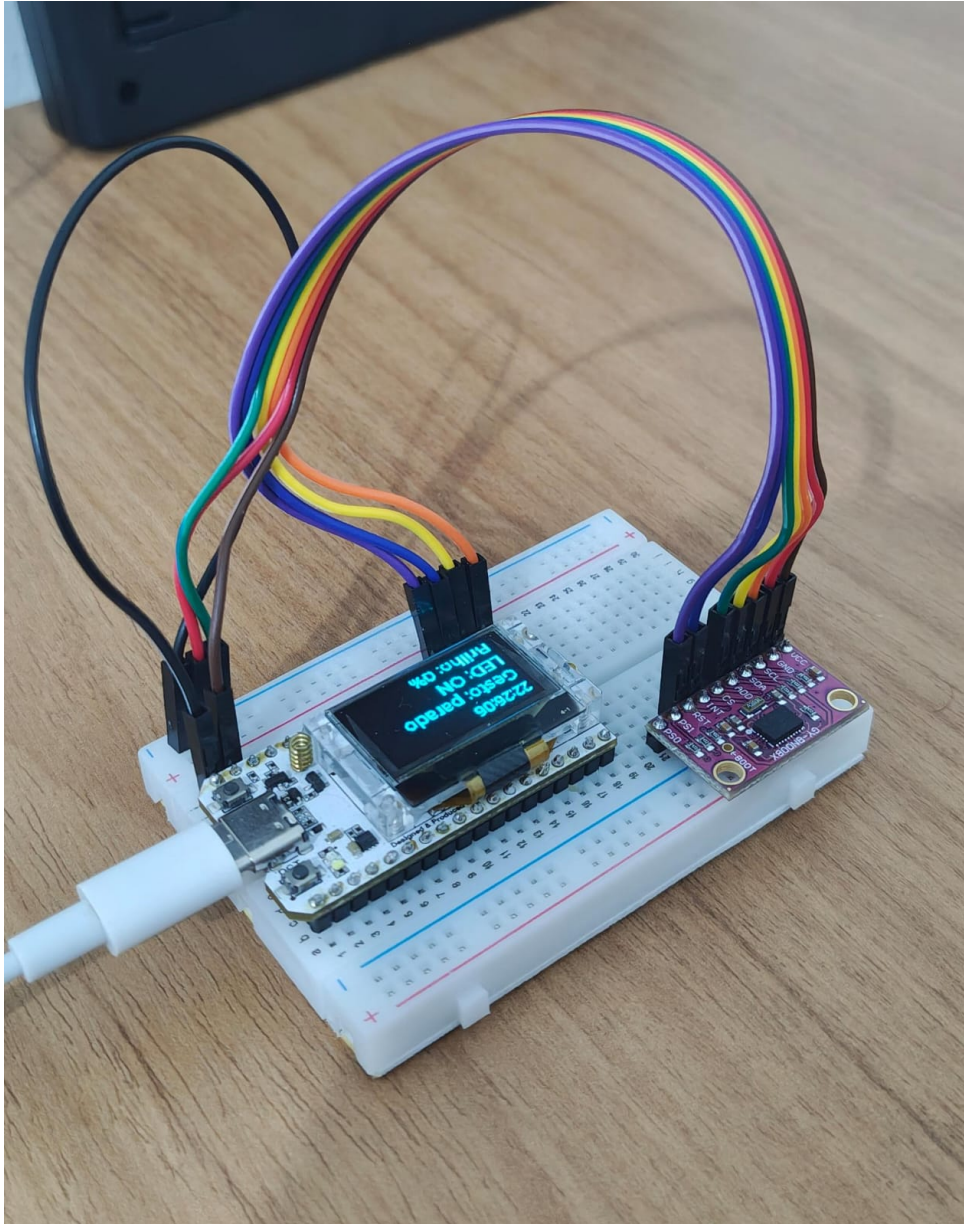


Figure 1: Photo of the real assembly with visible jumpers

Component	Function
i2c_config	Initializes both I2C buses + Vext rail
oled_setup	SSD1306 Panel + LVGL via <code>esp_lvgl_port</code>
bno085	Vendored driver over the SH2 library (the registry version had a %u bug that broke <code>-Werror</code>)
edge_impulse	Edge Impulse SDK + trained model (<code>model-parameters/</code> , <code>tflite-model/</code>)

3.2 Main Modules (main/)

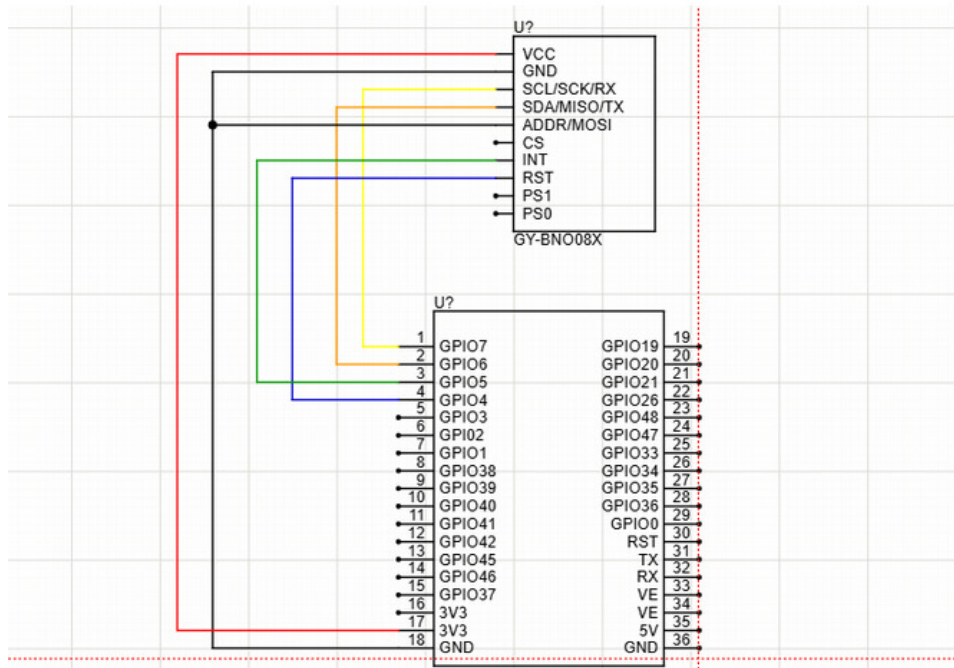


Figure 2: Schematic/Fritzing diagram of the BNO085 ↔ Heltec connection

File	Function
main.c	Orchestrates boot
imu_config.cpp	Core: PWM, sample windowing, inference, gesture trigger logic, OLED, clock, diagnostics
wifi_ntp.c	Wi-Fi STA + SNTP
mqtt_link.c	MQTT Client with FreeRTOS queue

3.3 Boot Sequence

Wi-Fi (non-blocking) → Vext rail → OLED → IMU → waits for IP (up to 30s) → MQTT.

3.4 Tasks

imu_service (priority 5), sensor callback, mqtt_pub, clock, diag, network, LVGL internal task.

3.5 Partitions (partitions.csv)

nvs 24K, phy_init 4K, factory 4 MB (binary $\approx$ 1.1 MB).

4 Machine Learning Pipeline

1. **Data collection:** a separate firmware (receber_dados) transmits CSV of roll, pitch, yaw at 50 Hz via the Edge Impulse Data Forwarder. The quaternion to Euler conversion formulas are identical in the collector and the final firmware:

```

1 roll = atan2f(2*(qr*qi + qj*qk), 1 - 2*(qi*qi + qj*qj)) * 57.2958f;
2 pitch = asinf(2*(qr*qj - qk*qi)) * 57.2958f;
3 yaw = atan2f(2*(qr*qk + qi*qj), 1 - 2*(qj*qj + qk*qk)) * 57.2958f;

```

Listing 1: Quaternion to Euler conversion

2. **Dataset:** 5 classes — **down, up, right, left, idle**.
3. **Model:** window of 150 samples $\times$ 3 axes = 450 values ($\approx 3s$), int8 quantized network, compiled with the EON Compiler.
4. **Inference:** buffer filled by the sensor callback; as soon as the window is full $\rightarrow$ `run_classifier()`. Configured via macros (`EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE`, `EI_CLASSIFIER_LABEL_COUNT`).
5. **Anti-repetition trigger:** an action is only triggered when the winning label changes **and** confidence $\geq 60\%$. Detecting idle rearms the trigger.
 - **right** $\rightarrow$ **idle** $\rightarrow$ **idle**: LED stays on
 - **right** $\rightarrow$ **idle** $\rightarrow$ **right**: no new action

IMAGE: Edge Impulse Studio screenshots — data explorer, accuracy/loss, confusion matrix
(expected file: docs/images/8.jpg)

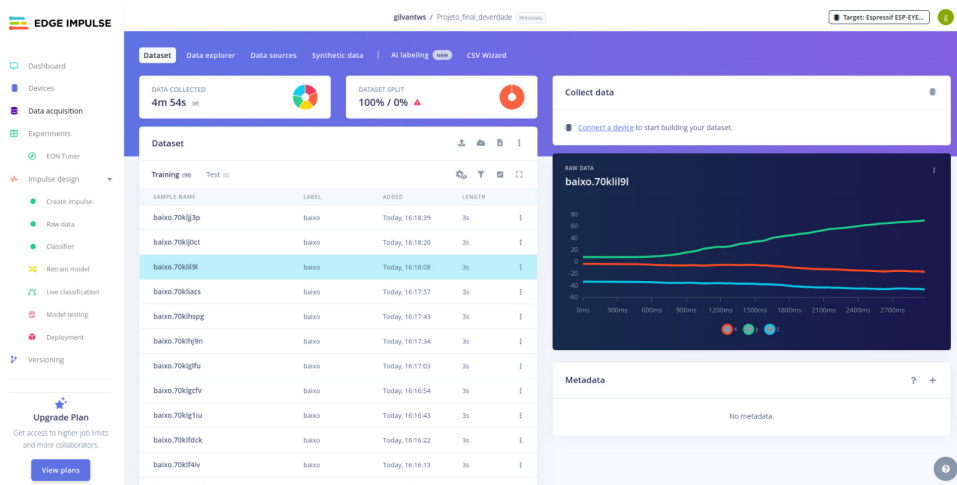


Figure 3: Graph of a gesture sample showing pitch/yaw variation

5 Training Details (Edge Impulse)

This section documents the exact configuration used in Edge Impulse Studio to generate the embedded model, ensuring experiment reproducibility.

5.1 Impulse Configuration (Windowing)

5.2 Signal Processing (DSP)

Raw Data block, with all 3 axes enabled and scaling factor (**Scale axes**) set to 1, meaning no additional normalization beyond what is already done in the quaternion $\rightarrow$ Euler conversion.

Parameter	Value
Input axes	x, y, z (3 axes)
Window size	3,000 ms
Window increase (stride)	1,000 ms
Frequency (Hz)	50 Hz
Zero-pad data	Enabled
Train on data subset	100%
Output classes	5 (down, up, right, left, idle)

Table 4: Parameters of the *Time series data* block

On-device performance metric	Value
Processing time	1 ms
Peak RAM usage	2 KB

Table 5: Estimated performance of the DSP block on the embedded device

5.3 Neural Network Architecture

Dense (fully-connected) neural network, trained and quantized in int8 by the EON Compiler.

Layer	Detail
Input layer	450 features (window of 150 samples $\times$ 3 axes)
Dense layer	20 neurons
Dense layer	10 neurons
Output layer	5 classes

Table 6: Neural network architecture

5.4 Results (Validation Set)

Note on results: 100% accuracy on the validation set is a positive sign, but it may also reflect a relatively small or non-diverse dataset (few users/collection conditions). It is recommended to validate the model with data collected in sessions and conditions different from those used in training, to confirm its generalization capability before considering it final.

6 LED Control via PWM

LEDC: channel 0, timer 0, low-speed mode, 5 kHz, 10 bits (duty 0-1023).

```
1 duty = (brightness_percent * 1023) / 100; // OFF = duty 0
```

Listing 2: Duty cycle calculation

7 OLED Display and NTP Clock

4 lines, displayed in this order:

Training parameter	Value
Number of training cycles	30
Learning rate	0.0005
Use learned optimizer	Disabled
Training processor	CPU
Model version	Quantized (int8)

Table 7: Training hyperparameters

Metric	Value
Accuracy	100.0%
Loss	0.03
Area under ROC Curve	1.00
Weighted average Precision	1.00
Weighted average Recall	1.00
Weighted average F1 score	1.00

Table 8: General model performance metrics

1. **Time** — clock in HH:MM:SS format
2. **Gesture** — last detected gesture
3. **LED** — on/off state
4. **Brightness** — current percentage

Updates on every classified window or remote command (shows origin, e.g., `remote:on`). Before synchronization, the time displays `--:--`.

Without a battery-backed RTC, the board always boots to the Unix epoch. Once connected to Wi-Fi, SNTP synchronizes with `pool.ntp.org` and `a.st1.ntp.br`, timezone `<-03>3` (Brasília, no daylight saving time).

8 IoT / MQTT Connectivity

Broker: `broker.hivemq.com:1883`

8.1 Publishing

Topic `testtopic/pnaat/status` (QoS 1), upon connection, on every gesture and remote command:

```

1 {
2   "source": "up",
3   "led": true,
4   "brightness": 80,
5   "time": "21:03:12"
6 }
```

Listing 3: Telemetry payload example

8.2 Subscribing

Topic `testtopic/pnaat/cmd`, payloads: `on`, `off`, `up`, `down`.

	DOWN	UP	RIGHT	LEFT	IDLE
DOWN	100%	0%	0%	0%	0%
UP	0%	100%	0%	0%	0%
RIGHT	0%	0%	100%	0%	0%
LEFT	0%	0%	0%	100%	0%
IDLE	0%	0%	0%	0%	100%

Table 9: Confusion matrix (validation set) — F1 score = 1.00 across all classes

Gesture	Action
right	LED ON
left	LED OFF
up	+20% brightness
down	-20% brightness

8.3 Hybrid Semantics

Last write wins — a remote command is applied immediately, but local gestures continue working afterwards.

8.4 Resilience

Publishing goes through a FreeRTOS queue outside the sensor's critical path; without Wi-Fi/internet, everything continues working locally, with automatic reconnection. Credentials configurable via `menuconfig` ("Configuracao Wi-Fi / MQTT (PNAAT)" section).

IMAGE: Screenshot of JSON telemetry arriving at an MQTT client
(expected file: docs/images/mqtt-telemetry.png)

IMAGE: Photo of the white LED turning on while the console sends the on command
(expected file: docs/images/mqtt-comando-on.jpg)



Figure 4: Classifier training screen with confusion matrix

9 Built-in Diagnostics

The `diag` task prints every 5s:

```
1 [diag] X samples/s | Y INT pulses/s | windows: Z
```

Expected: ≈ 50 samples/s and ≈ 100 pulses/s.

Samples	INT Pulses	Diagnostics
0	0	Wiring problem on INT
0	OK	Software problem
OK	OK	Healthy system

10 How to Build and Flash

```
1 source ~/esp/v5.4.4/esp-idf/export.sh
2 idf.py menuconfig          # fill in SSID/password in the PNAAT section
3 idf.py -p ttyUSB0 build flash monitor
```

Listing 4: Build and flash

Optional MQTT console in tools/:

```
1 ./tools/pnaat_mqtt.sh
```

Interactive terminal showing formatted telemetry (time | source | LED | brightness) and sending commands.

Relevant settings in sdkconfig: optimization -Os, CONFIG_PARTITION_TABLE_CUSTOM, 8 MB flash, LV_FONT_MONTSEERRAT_12 enabled.

11 Technical Challenges Solved

1. Link error: `imu_config_init()` declared/called but never defined.
2. Migration from legacy I2C API to `driver/i2c_master.h`.
3. Bug in the registry component `rinku404/bno085 (%u broke -Werror)` — fixed by vendoring the component.
4. Conflict between a custom flush callback and the internal callback of `esp_lvgl_port`.
5. Standard 1 MB partition overflow — fixed with custom 4 MB partition table and flash size correction in `sdkconfig` (2 MB configured vs. 8 MB actual).
6. Bloated binary due to `-Og` — changed to `-Os`, suppressing false positive `-Wuninitialized` warnings from Edge Impulse SDK headers.
7. Race condition in MQTT (DNS error `getaddrinfo 202`) — fixed by starting MQTT only after `GOT_IP`.
8. Intermittent IMU failures — root cause: loose jumper on INT pin, found via diagnostics.
9. GCC 14 deprecations (addition of `volatile`, `std::is_pod`).

12 Repository Structure

```
1 Projeto_finalPNAAT/
2 |-- build/
3 |-- components/
4 |   |-- i2c_config/
5 |   |-- oled_setup/
6 |   '-- bno085/                # vendored
7 |-- docs/
8 |   |-- images/                # images for this document
9 |   '-- documentacao.tex       # this file
10 |-- main/
11 |   |-- main.c
12 |   |-- imu_config.cpp
13 |   |-- imu_config.h
14 |   |-- wifi_ntp.c
15 |   |-- wifi_ntp.h
16 |   |-- mqtt_link.c
17 |   |-- mqtt_link.h
18 |   |-- Kconfig.projbuild
19 |   '-- idf_component.yml
20 |-- managed_components/
```

```
21 | -- tools/  
22 |   |-- pnaat_mqtt.sh  
23 |   '-- pnaat_mqtt.py  
24 | -- CMakeLists.txt  
25 | -- LICENSE  
26 | -- README.md  
27 | -- dependencies.lock  
28 | -- partitions.csv  
29 | -- sdkconfig  
30 |-- sdkconfig.old
```

13 Technologies Used

- ESP-IDF v5.4.4
- FreeRTOS
- Edge Impulse + TensorFlow Lite Micro (EON)
- LVGL 9.2
- esp_lvgl_port
- MQTT (esp-mqtt)
- SNTP/LwIP
- LEDC Driver
- New i2c_master API

14 Credits and Acknowledgments

PNAAT Course · Edge Impulse Platform · Heltec · Espressif Community

15 License

This project is licensed under the MIT License — see the `LICENSE` file for more details.

100% Edge AI — classification happens on the microcontroller, without cloud dependency.

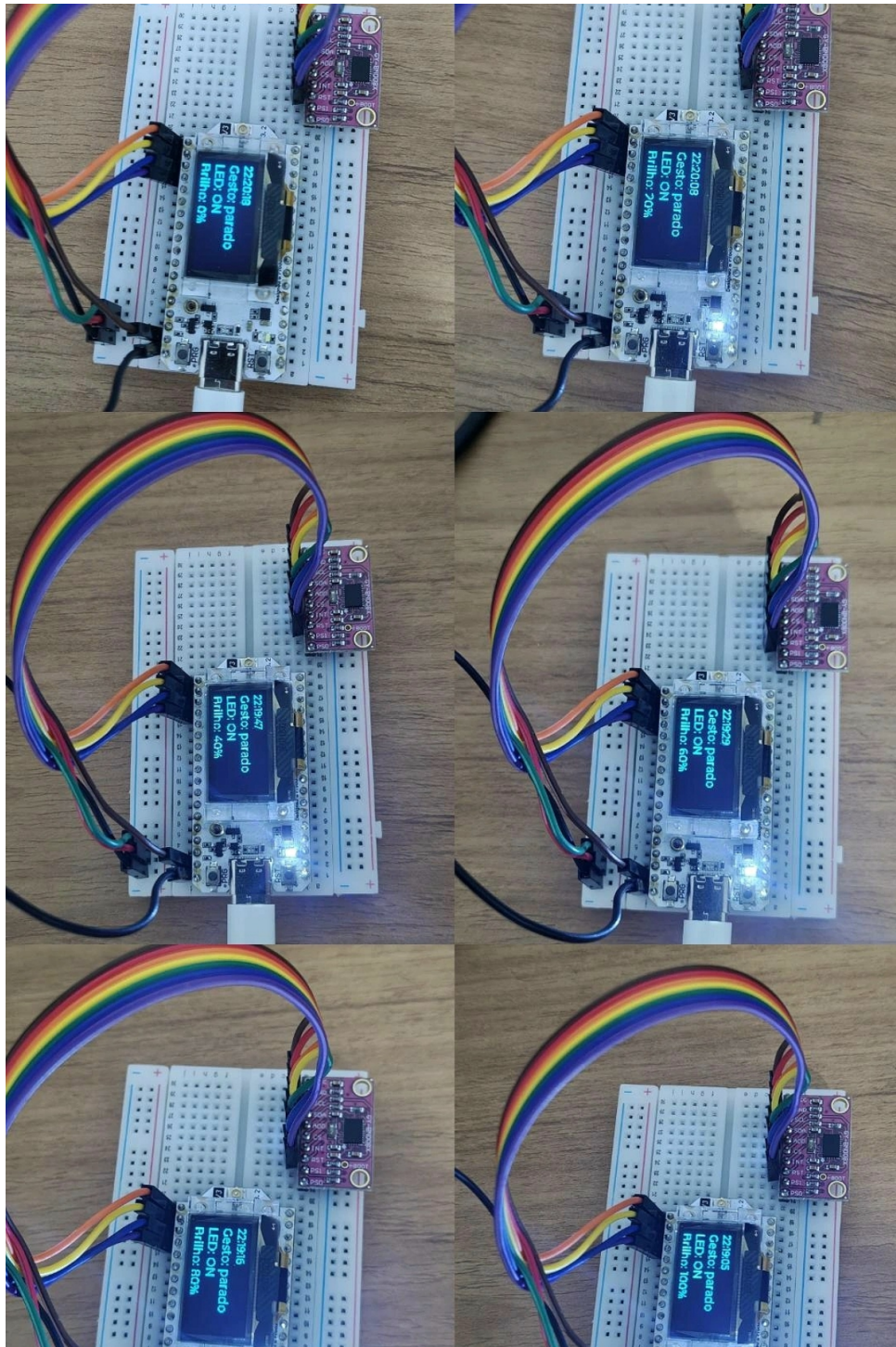


Figure 5: Sequence of photos of the white LED at 0%, 40%, and 80% brightness